

Construindo um Catálogo de Produtos com Vue 3 e CSS Grid

Este documento descreve as etapas necessárias para desenvolver um Catálogo de Produtos, organizando os cards em colunas por linha.

Etapa 1 — Criação da Aplicação

Crie uma pasta para a aplicação. Depois, abra esta pasta com o VSCode e abra o terminal integrado.

No terminal, execute:

```
npm init vue@latest .
```

Dentre as opções, escolha: ☐ Add TypeScript support? No ☐ Add JSX Support? No ☒ Add Vue Router for Single Page Application development? Yes ☐ Add Pinia for state management? No ☐ Add Vitest for Unit Testing? No ☒ Add an End-to-End Testing Solution? No ☒ Add ESLint for code quality? No

Depois de criada a aplicação, instale as dependências:

```
npm install
```

É então gerada uma estrutura inicial com alguns arquivos de exemplo que não serão utilizados. Você pode excluir os arquivos que são criados dentro das pastas `src/components/`, `src/views/` e `src/assets/` conforme necessário.

Etapa 2 — Simplificação do `App.vue`

O conteúdo do arquivo `App.vue` é simples nesta aplicação. Como toda a navegação é controlada pelo Vue Router, o `App.vue` apenas renderiza o componente correspondente à rota ativa através do `<RouterView />`:

```
<script setup>
import { RouterView } from 'vue-router'
</script>

<template>
  <RouterView />
</template>
```

Não há navegação manual entre páginas, cabeçalho ou rodapé nesta versão — o objetivo é manter o foco na exibição do catálogo.

No que respeita ao CSS, o `App.vue` não possui estilos próprios, já que a estilização é delegada aos componentes específicos (como `ProdutoCard.vue`) e à view principal (`CatalogoView.vue`).

Etapa 3 — Configuração do Vue Router

O roteador é configurado em `src/router/index.js`. A única rota definida aponta a raiz `/` para o componente `CatalogoView`, que é o coração da aplicação:

```
import { createRouter, createWebHistory } from 'vue-router'
import CatalogoView from '@/views/CatalogoView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: CatalogoView,
    },
  ],
})

export default router
```

O uso de `createWebHistory` faz com que a URL seja limpa (sem o `#`), aproveitando o History API do navegador. O alias `@` aponta para `src/`, o que torna os imports mais legíveis e independentes da profundidade do arquivo.

Etapa 4 — Criação dos dados dos produtos (`utils/produtos.js`)

Antes de criar qualquer componente visual, é necessário ter os dados que serão exibidos. O arquivo `src/utils/produtos.js` contém um array de objetos, onde cada objeto representa um produto do catálogo:

```
const produtos = [
  {
    id: 1,
    nome: 'Ração Premium Cães',
    preco: 120,
    categoria: 'Alimentos',
    imagem: '/images/racao-caes.png',
  },
  // ... demais produtos
]

export default produtos
```

Cada produto possui cinco propriedades:

Propriedade	Tipo	Descrição
<code>id</code>	Number	Identificador único
<code>nome</code>	String	Nome do produto
<code>preco</code>	Number	Preço em reais
<code>categoria</code>	String	Categoria a que o produto pertence
<code>imagem</code>	String	Caminho da imagem dentro de <code>public/</code>

Ao todo, são 9 produtos cadastrados, distribuídos entre as categorias Alimentos, Brinquedos, Higiene e Acessórios. As imagens ficam em `public/images/`, o que permite referenciá-las com caminhos absolutos a partir da raiz.

Aqui, as imagens utilizadas foram as mesmas da aplicação anteriormente desenvolvida em sala de aula.

Etapa 5 — Criação do componente `ProdutoCard.vue`

O componente `src/components/ProdutoCard.vue` é responsável por exibir as informações de um único produto. Ele recebe um objeto `produto` via prop e renderiza a imagem, o nome, a categoria e o preço formatado:

```
<script setup>
defineProps({
  produto: {
    type: Object,
    required: true,
  },
})
</script>

<template>
  <div class="produto-card">
    
    <h3>{{ produto.nome }}</h3>
    <p>{{ produto.categoria }}</p>
    <strong> R$ {{ produto.preco.toFixed(2) }} </strong>
  </div>
</template>

<style scoped>
.produto-card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  text-align: center;
}
```

```
}

.produto-imagem {
  width: 100%;
  height: 200px;
  object-fit: cover;
}
</style>
```

Alguns detalhes importantes:

- **defineProps** declara que o componente espera receber um objeto do tipo **Object**, marcado como obrigatório (**required: true**). Isso garante que o Vue emita um aviso no console caso o card seja usado sem receber a prop.
- **produto.preco.toFixed(2)** formata o número com sempre duas casas decimais, garantindo a exibição correta de valores como **R\$ 120.00**. Lembre-se de que nós já temos a função para formatação de moeda desenvolvida na aplicação anterior, mas aqui optamos por uma solução mais simples para manter o foco no layout.
- **object-fit: cover** na imagem faz com que ela preencha o espaço de **200px** de altura sem distorcer a proporção original — partes da imagem podem ser cortadas, mas o enquadramento fica uniforme entre todos os cards.
- O uso de **<style scoped>** garante que os estilos se apliquem apenas aos elementos deste componente, sem vazarem para o restante da aplicação.

Etapa 6 — Construção da **CatalogoView.vue** com CSS Grid

Esta é a etapa central da aplicação. O arquivo **src/views/CatalogoView.vue** é a view que reúne todos os cards e os organiza visualmente usando CSS Grid:

```
<script setup>
import ProdutoCard from '@components/ProdutoCard.vue'
import produtos from '@utils/produtos'
</script>

<template>
  <div class="catalogo">
    <ProdutoCard v-for="produto in produtos" :key="produto.id"
:produto="produto" />
  </div>
</template>

<style scoped>
.catalogo {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 24px;
}
</style>
```

Como o `v-for` popula o catálogo

A diretiva `v-for="produto in produtos"` itera sobre o array importado de `utils/produtos.js` e renderiza um `<ProdutoCard>` para cada item. O atributo `:key="produto.id"` fornece ao Vue um identificador único por elemento, o que permite que o *framework* gerencie o DOM de forma eficiente ao reordenar ou atualizar a lista. A prop `:produto="produto"` passa o objeto completo do produto para dentro do componente filho. Nas aulas ainda não chegamos a utilizar um `Object` como prop, mas isso é perfeitamente válido e comum em Vue — o componente `ProdutoCard` é projetado para receber um objeto com as propriedades necessárias para exibir as informações do produto. O resultado final é o mesmo se utilizássemos as propriedades individualmente (como `:nome="produto.nome"`), mas passar o objeto inteiro é mais prático e mantém o código mais limpo, especialmente quando há várias propriedades.

A classe `.catalogo` e o CSS Grid

A classe `.catalogo` é o container Grid. É aqui que reside a lógica de quantos cards aparecem por linha:

```
.catalogo {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 24px;  
}
```

Cada propriedade tem um papel específico:

`display: grid`

Transforma o elemento `<div class="catalogo">` em um **Grid Container**. A partir desse momento, todos os seus filhos diretos — os `<ProdutoCard>` gerados pelo `v-for` — tornam-se **Grid Items** e passam a ser posicionados de acordo com as regras definidas no container.

`grid-template-columns: repeat(3, 1fr)`

Esta é a propriedade que define **quantos cards aparecem em cada linha**. Ela instrui o Grid a criar exatamente **3 colunas**, cada uma ocupando uma fração igual (**1fr**) do espaço disponível.

A função `repeat(n, tamanho)` é um atalho para evitar repetição. Escrever `repeat(3, 1fr)` é equivalente a escrever `1fr 1fr 1fr`, que por sua vez significa: divida o espaço total do container em 3 partes iguais e cada coluna ocupa uma delas.

A unidade **fr** (fraction) é relativa ao espaço disponível após descontar as lacunas (**gap**). Se o container tiver, por exemplo, **900px** de largura e **gap: 24px**, o cálculo seria aproximadamente:

```
Espaço disponível = 900px - (2 × 24px) = 852px  
Largura de cada coluna = 852px ÷ 3 = 284px
```

Com 3 colunas definidas e 9 produtos no array, o Grid posiciona automaticamente **3 cards por linha**, resultando em 3 linhas completas.

Se o valor fosse alterado para `repeat(4, 1fr)`, haveria 4 cards na primeira linha e 4 na segunda, com 1 card sozinho na terceira. Para `repeat(2, 1fr)`, seriam 2 cards por linha e 5 linhas no total (4 completas + 1 com apenas 1 card). O número de linhas **não precisa ser declarado** — o Grid cria quantas forem necessárias automaticamente.

gap: 24px

Define o espaçamento entre os cards, tanto nas direções horizontal (entre colunas) quanto vertical (entre linhas). É um atalho para `row-gap` e `column-gap` ao mesmo tempo. O valor de `24px` garante que este espaçamento seja consistente, independentemente do número de colunas ou do tamanho dos cards.
